

# ARE 212 SECTION 14: Replication

Fiona Burlig

April 21, 2016

It's our final section! This semester has flown by. To cap everything off, we're going to attempt to replicate (most of) a really cool paper that came out in the *American Economic Review* last year: [Michael Callen](#) and [James Long](#)'s "Institutional Corruption and Election Fraud: Evidence from a Field Experiment in Afghanistan."<sup>1</sup> Why are we spending our last section replicating a paper? First of all, we'll get to use several of the skills that we've been building up over the course of the semester, which is always a nice way to wrap up a class. Secondly, making sure your own work is replicable is a really important part of being a good researcher - and top journals are increasingly asking for replication archives with paper submissions.<sup>2</sup> Finally, I can't think of a better way to send you off into the big wide world of econometrics than by talking about real research. Of course, it wouldn't be ARE 212 section without me sneaking a new topic or two in: we're going to also briefly touch on randomization inference and Lee bounds. We've got a lot to do, so let's get to it!

## Off we go

I don't necessarily expect that you've read the paper, so let's start by briefly discussing what the paper is about and where we're going. The best first step for this is to read the abstract. The authors write:

*We investigate the relationship between political networks, weak institutions, and election fraud during the 2010 parliamentary election in Afghanistan combining: (i) data on political connections between candidates and election officials; (ii) a nationwide controlled evaluation of a novel monitoring technology; and (iii) direct measurements of aggregation fraud. We find considerable evidence of aggregation fraud in favor of connected candidates and that the announcement of a new monitoring technology reduced theft of election materials by about 60 percent and vote counts for connected candidates by about 25 percent. The results have implications for electoral competition and are potentially actionable for policymakers.*

Even before we dive in, it's maybe not surprising after reading this abstract that the paper is in our discipline's top journal: the authors have deployed an RCT in a contentious context, and find that it has the potential to substantially improve the exercise of democracy. Awesome.

The key ideas of this paper are the following: elections in Afghanistan (and many other developing countries) are likely subject to election fraud.<sup>3</sup> In particular, these authors investigate what they call *aggregation fraud*: fraud

---

<sup>1</sup>The paper can be found [here](#), along with the data replication archive we're drawing from today.

<sup>2</sup>From personal experience, you'll do yourself a big favor if you write replicable code!

<sup>3</sup>There are lots of institutional reasons for election fraud being likely in Afghanistan; the paper has more details on this, but the basic idea is that seats are often won or lost by a razor-thin margin, so the returns to modifying vote totals is very high. As you might expect, enforcement is also present, but likely limited.

that occurs between votes being tallied at individual polling centers and the aggregated tallies that reach the central polling authority. In Afghanistan, polling center-level vote tallies are supposed to be posted outside the polling station immediately following the election. Savvy researchers can then compare these totals to the official records posted online for these polling stations to get a measure of fraud. Our authors will pull this off by taking photos of the posted polling station vote sheets, and comparing these to official records acquired by web scraping. So the research questions are: Does this fraud follow a clear pattern? And is there an easy way of reducing fraud of this kind?

This paper is going to do a couple of things: first, it will document widespread election fraud. This is important, because if there were no evidence of election fraud, then there'd be no point in implementing the experiment. Seems obvious, but having good motivation is like a quarter of the battle of doing good research. In doing this, the authors will also look at whether connections between candidates and election officials change the observed level of fraud. We'll use the results in this section to introduce a new technique for hypothesis testing: randomization inference.

Next, they'll tackle the second question, with a really neat experiment. The research team sent letters to randomly-selected polling places telling election officials that they would be coming by the day after the election to photograph the official vote tally sheets. They then look at whether this affected the discrepancy between the local sheets and the official records. (Spoiler alert: yes!) This section of the paper also introduces a cool empirical technique that's worth being aware of: Lee bounds.

Finally, they'll conclude by doing some robustness and additional analysis. We'll skip this in favor of spending a little bit more time on the main results, but they're worth checking out if you haven't already.

With a little bit of a road-map in mind, let's get going.<sup>4</sup>

## I am shocked – shocked!

The first thing the authors need to do is to demonstrate that aggregation fraud is present in their sample. We'll start by doing the same. First, let's load our usual packages:

```
library(haven)
library(dplyr)
library(xtable)
library(ggplot2)

##### FUNCTIONS
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

tbl_dfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
```

---

<sup>4</sup>We're not going to completely replicate every result in the paper in the interest of time and focusing on what's really useful, but we'll cover the highlights.

```

    as.matrix()
}

OLS <- function(data, y, X) {
  n <- nrow(data)
  k <- length(X)
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  # beta
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  # resids
  e <- ydata - xdata %*% betahat
  s2 <- (t(e) %*% e) / (n-k)
  XpXinv <- solve(t(xdata) %*% xdata)
  #std error
  se <- sqrt(s2 * diag(XpXinv))
  # t statistic & its pvalue
  tStat <- (betahat - 0) / se
  pVal <- tStat %>%
    abs() %>%
    "*"(-1) %>%
    pt(df = n - k) %>%
    "*" (2)
  olsOut <- list(betahat, se, pVal) %>%
    as.tbl_df()
  names(olsOut) <- c("betahat", "SE", "tStatPVal")
  return(olsOut)
}

miniOLS <- function(data, y, X) {
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  # beta
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  return(betahat)
}

# function we wrote two weeks ago
randomizer <- function(data, p) {
  # randomly sample p % of the rows in your dataset
  randomObs <- 1:nrow(data) %>%
    sample(size = nrow(data) * p)
  # create a treatment variable = 1 if your row was sampled
  data <- mutate(data,
    t = ifelse(row_number() %in% randomObs, 1, 0))
  return(data$t)
}

```

```
##### RANDOMIZATION SEED
set.seed(12345)

##### GGLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
  panel.border = element_rect(fill = NA, color = "black"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_line(color = "gray5"),
  axis.text = element_text(color = "black", size = 10),
  axis.title = element_text(color = "black", size = 12),
  legend.key = element_blank()
)
```

I've included the full replication file for the paper, including the dataset, do files, and readme file, in the Section 14 folder on bCourses. Requiring authors to submit replication data before publication is a relatively new practice, so there aren't really established norms on how these files have to be coded or organized. If you start looking at these archives (and I encourage you to do so!), you'll see vast heterogeneity in what's provided. In my mind, best practice is to have a replication archive start with completely raw data, and end up reproducing the final tables and figures that are available in the paper.<sup>5</sup> It's also very common to just provide final analysis datasets and programs that run regressions and make figures off of these datasets. I'm interested to see what norms surrounding replication archives will evolve to be.<sup>6</sup>

The Callen and Long file has four datasets, of which we'll use two: `master_pc_native.dta` (this has pre-intervention characteristics of polling locations) and `master_vote_difference_native.dta` (this includes everything we'll need for the main tables).<sup>7</sup> It also has a ton of do-files - each one of which makes a table or figure in the paper or appendix. I personally like to have one master file that calls everything, but like I said, to each her own.<sup>8</sup> Note that everything in this replication archive is, like most papers in economics these days, in Stata format. Even if you're an R afficianado, it pays to be able to speak Stata.

We're going to start off by trying to replicate Table 1 in the paper. This table is all about the types of aggregation fraud that are going on in the sampled polling stations, and looks like this:<sup>9</sup>

---

<sup>5</sup>There are legitimate reasons for not doing this, though: for example, raw data are often confidential. In this case, authors will usually provide code and instructions for obtaining the raw data, but don't include the datasets themselves. Young authors also often want to protect the property rights associated with a dataset that was difficult to collect; providing the full raw dataset often entails providing *all* the variables in that dataset, which, if you're intent on publishing a few more papers out of the same data, might not always be desirable. Strive to strike a balance between research transparency and practicality.

<sup>6</sup>It's not currently common practice in economics for editors to actually *run* the code in your replication archives before publication - the time costs to this are large. But it's always super frustrating for other researchers when your code isn't implementable, so please do make sure that your replication works - for your own sake and for everybody else's.

<sup>7</sup>`master_ps_native.dta` has information on polling substations; and `di_candidate_information.dta` is background demographic information on candidates.

<sup>8</sup>This replication file, like most, has a readme file. Like many, it isn't particularly informative.

<sup>9</sup>I am committing an egregious sin by not copying the notes. Smite me, paper Gods!

TABLE 1—AGGREGATION DISCREPANCY PATTERNS

| Pattern                        | Number of polling substations<br>(1) | Share of sample<br>(2) | Mean vote difference<br>(3) |
|--------------------------------|--------------------------------------|------------------------|-----------------------------|
| No fraud                       | 74                                   | 21.4 percent           | 0.00                        |
| Adding votes only              | 70                                   | 20.2 percent           | 47.34                       |
| Subtracting votes only         | 15                                   | 4.3 percent            | −245.07                     |
| Adding and subtracting equally | 15                                   | 4.3 percent            | 0.00                        |
| Adding more than subtracting   | 127                                  | 36.7 percent           | 83.45                       |
| Subtracting more than adding   | 45                                   | 13.01 percent          | −54.13                      |

To do this, we'll need `master_vote_difference_native.dta`, so let's bring it in:

```
mvdn <- read_dta("master_vote_difference_native.dta")
# and look at it
mvdn

## Source: local data frame [386,951 x 39]
##
##      pc provid398 treat_lib strata_group picture_votes_1st
##      (dbl)      (dbl)      (dbl)      (dbl)      (int)
## 1  101025          1          0          1          NA
## 2  101025          1          0          1          NA
## 3  101025          1          0          1          NA
## 4  101025          1          0          1          NA
## 5  101025          1          0          1          NA
## 6  101025          1          0          1          NA
## 7  101025          1          0          1          NA
## 8  101025          1          0          1          NA
## 9  101025          1          0          1          NA
## 10 101025          1          0          1          NA
## ..      ...      ...      ...      ...
## Variables not shown: cand_id_s (chr), votes (int), vote_diff
##      (dbl), pollingstation (int), cand_data (dbl), pashtun (dbl),
##      tajik (dbl), treat_di_actual (int), within_1k (int), abs_diff
##      (dbl), pos_diff (dbl), neg_diff (dbl), incumbent (dbl),
##      powerful (dbl), peo_fullsample (dbl), deo_fullsample (dbl),
##      karzai_fullsample (dbl), gov_fullsample (dbl), kt (dbl), gt
##      (dbl), pt (dbl), dt (dbl), pflt (dbl), after_total (dbl),
##      treat_1k_dummy2 (int), treat_1k_dummy3 (int), treat_1k_dummy4
##      (int), treat_1k_dummy5 (int), treat_1k_dummy6 (int),
##      within_1_2k (dbl), treat_1k_dum (dbl), treat_1_2k_dum (dbl),
##      completesample (dbl), winpos (dbl)
```

That doesn't really tell us much about what's actually going on here - the variables are (of course) unlabeled. But we can figure this out. It's probably a good idea to take a look at the whole dataset:

```
View(mvdn)
```

From scrolling through, it becomes clear that these data are at the polling station-candidate level - *not* at the polling station level. We need data at the polling station level in order to compute the things we want.

The very first thing we'll need to do is remove the polling stations where there's no data, either because the research team didn't collect a photo or because they couldn't scrape data for this station. We could filter out these rows one variable at a time, or just use the conveniently included `vote_diff` variable. We want all of the rows for which the vote difference *isn't* missing:<sup>10</sup>

```
mvdn <- filter(mvdn, is.na(vote_diff) == FALSE)
mvdn

## Source: local data frame [48,018 x 39]
##
##      pc provid398 treat_lib strata_group picture_votes_1st
##      (dbl)      (dbl)      (dbl)      (dbl)      (int)
## 1  101025          1          0          1          20
## 2  101025          1          0          1           1
## 3  101025          1          0          1           0
## 4  101025          1          0          1           0
## 5  101025          1          0          1          13
## 6  101025          1          0          1           0
## 7  101025          1          0          1           4
## 8  101025          1          0          1           0
## 9  101025          1          0          1           0
## 10 101025          1          0          1           6
## ..      ...      ...      ...      ...
## Variables not shown: cand_id_s (chr), votes (int), vote_diff
##      (dbl), pollingstation (int), cand_data (dbl), pashtun (dbl),
##      tajik (dbl), treat_di_actual (int), within_1k (int), abs_diff
##      (dbl), pos_diff (dbl), neg_diff (dbl), incumbent (dbl),
##      powerful (dbl), peo_fullsample (dbl), deo_fullsample (dbl),
##      karzai_fullsample (dbl), gov_fullsample (dbl), kt (dbl), gt
##      (dbl), pt (dbl), dt (dbl), pflt (dbl), after_total (dbl),
##      treat_1k_dummy2 (int), treat_1k_dummy3 (int), treat_1k_dummy4
##      (int), treat_1k_dummy5 (int), treat_1k_dummy6 (int),
##      within_1_2k (dbl), treat_1k_dum (dbl), treat_1_2k_dum (dbl),
##      completesample (dbl), winpos (dbl)
```

We've lost a ton of observations - good!<sup>11</sup> How many polling stations are left in our sample? We can use `dplyr`'s `distinct()` function to figure this out,

```
psNr <- dim(distinct(mvdn, pc, pollingstation))[1]
psNr
```

<sup>10</sup>This gives you the same polling stations as if we'd calculated this one variable at a time.

<sup>11</sup>In that it makes our lives easier, and in that our filtering worked...

```
## [1] 346
```

This matches the paper - good.<sup>12</sup>

We can also get rid of a bunch of variables - we only need a couple for this part of the exercise:

```
mvdn <- select(mvdn,
               pc, pollingstation, vote_diff)
```

Much better. Now we can create variables that tell us how many votes were adjusted in the positive direction, and how many in the negative direction:

```
mvdn <- mutate(mvdn,
               # if the vote difference was positive, what was it
               posVotes = ifelse(vote_diff > 0, vote_diff, 0),
               # if the vote difference was negative, what was it
               negVotes = ifelse(vote_diff < 0, vote_diff, 0))
```

This is all still at the polling station-candidate level. We can use two new `dplyr()` functions, `group_by()` and `summarize_each()`, to get everything at the polling station level. `group_by()` tells `dplyr` to treat all of the rows corresponding to one group of a dataset together, much like `-egen, group()-` in Stata. `summarize_each()` allows us to perform summary functions on multiple variables in a dataset. Unsurprisingly, combining these two functions allows us to perform summary functions on multiple variables *within each group* of a grouped dataset:

```
mvdnPS <- group_by(mvdn, pc, pollingstation) %>%
  summarize_each(funs(sum), vote_diff, posVotes, negVotes)
```

This dataset should have 346 rows.

```
mvdnPS

## Source: local data frame [346 x 5]
## Groups: pc [?]
##
##      pc pollingstation vote_diff posVotes negVotes
##      (dbl)          (int)      (dbl)    (dbl)    (dbl)
## 1  101025             2        150      163      -13
## 2  101025             4        137      158      -21
## 3  101033             1        169      199      -30
## 4  101033             2         -2        62      -64
## 5  101033             3        113      215     -102
## 6  101033             4         43        70      -27
## 7  101033             5        101      105       -4
## 8  101034             1         66        93      -27
## 9  101034             2         38        64      -26
## 10 101034             3         43        59      -16
## .. ...             ...         ...         ...         ...
```

---

<sup>12</sup>The table notes to Table 1 contain a typo. These notes say there's a sample of 341; this is actually 346.

Et voila.

Now we can create indicator variables for each type of election fraud, and then transform it into a factor variable:

```
mvdnPS <- mutate(mvdnPS,
  # create the absolute value of negative votes
  absNeg = abs(negVotes),
  # no fraud: + = - = 0
  # we set these as 1, 2, 3, 4, 5, 6 for each category
  # for easy transformation to factor later
  noFraud = ifelse((posVotes == absNeg) & (absNeg == 0), 1, 0),
  # + only
  addOnly = ifelse((posVotes > 0) & (absNeg == 0), 2, 0),
  # - only
  subtrOnly = ifelse((posVotes == 0) & (absNeg > 0), 3, 0),
  # + = 0 != 0
  addSubtrEq = ifelse((posVotes == absNeg) & (absNeg > 0), 4, 0),
  # + > - > 0
  addMore = ifelse((posVotes > absNeg) & (absNeg > 0), 5, 0),
  # - > + > 0
  subtrMore = ifelse((absNeg > posVotes) & (posVotes > 0), 6, 0)) %>%
# create a counter
mutate(ones = 1,
# create a factor variable that indicates which pattern we see
# notice that we can do this by adding because the categories are
# mutually exclusive
  pattern = noFraud + addOnly + subtrOnly + addSubtrEq + addMore + subtrMore,
  pattern = factor(pattern, levels = as.character(c(1:6)),
    labels = c("noFraud", "addOnly", "subtrOnly",
      "addSubtrEq", "addMore", "subtrMore"))) %>%
# clean up - only keep the vars we need
select(pc, pollingstation, vote_diff, pattern, ones)

head(mvdnPS$pattern)

## [1] addMore addMore addMore subtrMore addMore addMore
## 6 Levels: noFraud addOnly subtrOnly addSubtrEq ... subtrMore
```

We need to do one last thing before we can calculate all of the numbers in Table 1: we actually want our outcome data to be at the *pattern* level. We want to calculate the number of polling stations with each pattern, the percent of polling stations with each pattern, and the average vote difference for polling stations that display a given pattern. Here we go:

```
mvdnPattern <- group_by(mvdnPS, pattern)

# calculate the number & % of PS's by pattern
psNums <- summarize_each(mvdnPattern, funs(sum), ones) %>%
```



```

mutate(shareofsamp = ones / psNr)

# calculate the avg vote diff by pattern
psDiff <- summarize_each(mvdmPattern, funs(mean), vote_diff)

# and merge them back together
table1 <- left_join(psNums, psDiff, by = "pattern")

table1

## Source: local data frame [6 x 4]
##
##      pattern  ones shareofsamp  vote_diff
##      (fctr) (dbl)      (dbl)      (dbl)
## 1  noFraud    74   0.2138728    0.00000
## 2  addOnly    70   0.2023121   47.34286
## 3  subtrOnly  15   0.0433526  -245.06667
## 4 addSubtrEq  15   0.0433526    0.00000
## 5  addMore   127   0.3670520   83.44882
## 6  subtrMore  45   0.1300578  -54.13333

```

Winning!

## You get a car, you don't get a car, you get a car...

Time for our randomized controlled trial.<sup>13</sup> Now that we've shown that vote aggregation fraud is an issue in Afghanistan, we want to do something about it. Callen and Long are going to implement their experiment of sending letters notifying randomly-selected election officials that they'll be photographing vote counts. The first thing they need to do is to show that their randomization worked, and that there are no significant differences on baseline characteristics between treatment and control. To do this, they simply need to do a *t*-test between treatment and control for each characteristic. They do so in Table 4, which we'll replicate here.

---

<sup>13</sup>RCT-Oprah wouldn't be very popular, but would be kind of awesome.

TABLE 4—RANDOMIZATION VERIFICATION

|                                                               | Control<br>(1)   | Treat<br>(2)     | T-C<br>(3)        | p-value<br>(4) | Obs.<br>(Control)<br>(5) | Obs.<br>(Treat)<br>(6) |
|---------------------------------------------------------------|------------------|------------------|-------------------|----------------|--------------------------|------------------------|
| <i>Baseline survey data</i>                                   |                  |                  |                   |                |                          |                        |
| Plans to turnout during election (=1)                         | 0.790<br>[0.236] | 0.797<br>[0.232] | 0.008<br>(0.022)  | 0.734          | 223                      | 227                    |
| Believes vote is secret (=1)                                  | 0.668<br>[0.265] | 0.650<br>[0.255] | -0.018<br>(0.024) | 0.453          | 223                      | 227                    |
| Candidate will know how I voted (=1)                          | 0.086<br>[0.146] | 0.090<br>[0.153] | 0.004<br>(0.014)  | 0.767          | 223                      | 227                    |
| Can identify sitting MP (=1)                                  | 0.371<br>[0.325] | 0.386<br>[0.318] | 0.015<br>(0.030)  | 0.627          | 223                      | 227                    |
| People in precinct will vote<br>for same candidate (=1)       | 0.236<br>[0.253] | 0.249<br>[0.258] | 0.012<br>(0.024)  | 0.607          | 221                      | 224                    |
| Problems with ballot transport<br>are likely (=1)             | 0.538<br>[0.305] | 0.534<br>[0.302] | -0.004<br>(0.029) | 0.880          | 219                      | 225                    |
| Police in PC help security (=1)                               | 0.728<br>[0.246] | 0.737<br>[0.241] | 0.009<br>(0.023)  | 0.701          | 223                      | 227                    |
| People like you are threatened to<br>vote one way (=1)        | 0.213<br>[0.231] | 0.202<br>[0.223] | -0.011<br>(0.021) | 0.606          | 223                      | 226                    |
| Local violence likely on<br>election day (=1)                 | 0.500<br>[0.316] | 0.483<br>[0.347] | -0.017<br>(0.031) | 0.592          | 219                      | 226                    |
| MP Candidate from same Qawm (=1)                              | 0.235<br>[0.219] | 0.232<br>[0.227] | -0.003<br>(0.021) | 0.878          | 223                      | 227                    |
| Trad. auth. helps settle disputes (=1)                        | 0.286<br>[0.267] | 0.293<br>[0.240] | 0.007<br>(0.024)  | 0.785          | 223                      | 227                    |
| Pashtun (=1)                                                  | 0.323<br>[0.388] | 0.318<br>[0.407] | -0.005<br>(0.037) | 0.903          | 223                      | 227                    |
| Tajik (=1)                                                    | 0.436<br>[0.387] | 0.433<br>[0.390] | -0.004<br>(0.037) | 0.921          | 223                      | 227                    |
| Income generating activity (=1)                               | 0.602<br>[0.197] | 0.607<br>[0.192] | 0.004<br>(0.018)  | 0.817          | 223                      | 227                    |
| Electrified (=1)                                              | 0.722<br>[0.300] | 0.706<br>[0.323] | -0.016<br>(0.029) | 0.595          | 223                      | 227                    |
| District Governor keeps<br>election fair (=1)                 | 0.112<br>[0.170] | 0.114<br>[0.169] | 0.003<br>(0.016)  | 0.864          | 223                      | 225                    |
| <i>Administrative data</i>                                    |                  |                  |                   |                |                          |                        |
| Visited by international<br>election monitors (=1)            | 0.146<br>[0.353] | 0.182<br>[0.385] | 0.036<br>(0.035)  | 0.308          | 223                      | 227                    |
| Visited by domestic<br>election monitors (=1)                 | 0.892<br>[0.311] | 0.863<br>[0.344] | -0.029<br>(0.031) | 0.350          | 223                      | 227                    |
| Indelible ink washes<br>or not available (=1) (pre-treatment) | 0.798<br>[0.402] | 0.762<br>[0.427] | -0.036<br>(0.039) | 0.357          | 223                      | 227                    |

The first thing we need is the baseline characteristic data:

```
baseline <- read_dta("master_pc_native.dta") %>%
  # only keep what we need
  select(pc, treat_lib, expectedturnout, secretvote, candknowsvote, parliamentaryknowledge,
    onecandidatevote, transportproblems, policesafe, threatvote,
    violenceown, ownqawmcandidate, traditionalauthority,
    pashtun, tajik, employment, electrified, dgkeepsfair)
baseline
```

```
## Source: local data frame [471 x 18]
##
##      pc treat_lib expectedturnout secretvote candknowsvote
##      (dbl)      (dbl)          (dbl)      (dbl)          (dbl)
## 1  101025          0          0.6250000  0.3750000      0.1250000
## 2  101027          0          0.6666667  0.6666667      0.0000000
## 3  101028          1             NA          NA          NA
## 4  101031          1             NA          NA          NA
## 5  101032          0          0.7500000  0.3333333      0.0000000
## 6  101033          1          0.2500000  0.8750000      0.0000000
## 7  101034          1          0.6250000  0.6250000      0.0000000
## 8  101035          1          0.6666667  0.5000000      0.3333333
## 9  101037          0             NA          NA          NA
## 10 101038          0          0.4000000  0.5000000      0.0000000
## ..      ...      ...      ...      ...      ...
## Variables not shown: parliamentaryknowledge (dbl),
##   onecandidatevote (dbl), transportproblems (dbl), policesafe
##   (dbl), threatvote (dbl), violenceown (dbl), ownqawmcandidate
##   (dbl), traditionalauthority (dbl), pashtun (dbl), tajik (dbl),
##   employment (dbl), electrified (dbl), dgkeepsfair (dbl)
```

Our treatment indicator is `treat_lib` (usefully named, I know). To build up Table 4, we'd like to do a *t*-test between treatment and control for each covariate. What's the easiest way to do a *t*-test? Run a regression, of course! But we want to run a bunch of regressions - so obviously we're going to use a function from the `apply` family. Our table won't end up looking identical to Table 4 in the paper, because formatting is a pain, but we'll get close. We'll start by writing a function that will calculate the control mean, control SD, treatment mean, treatment SD, treatment minus control difference, standard error on the difference, *p*-value on the difference, number of observations in the treatment group, and number of observations in the control group. We'll spit out one row of a `tbl_df`. We'll then apply over this function.

```
tbl4Fxn <- function(data, depvar, treatvar) {
  myData <- select_(data, depvar, treatvar) %>%
    na.omit() %>%
    mutate(ones = 1)
  # group our data t vs. c
  grouped <- group_by_(myData, treatvar)
  # calculate mean and SD for each group
  meansd <- summarize_each_(grouped, funs(mean, sd), depvar)
  # count obs in each group
  counts <- summarize_each_(grouped, funs(sum), "ones")
  olsOut <- (OLS(myData, depvar, c("ones", treatvar)))[2,]

  finalOut <- c(depvar, meansd[1,2:3], meansd[2,2:3],
    olsOut, counts[1,2], counts[2,2]) %>%
    as.tbl_df()
  names(finalOut) <- c("variable", "C mean", "C SD", "T mean", "T SD",
    "T-C", "SE", "p-value", "C obs", "T obs")
}
```

```

return(finalOut)
}

```

Here we go!

```

variables <- c("expectedturnout", "secretvote", "candknowsvote", "parliamentaryknowledge",
               "onecandidatevote", "transportproblems", "policesafe", "threatvote",
               "violenceown", "ownqawmcandidate", "traditionalauthority",
               "pashtun", "tajik", "employment", "electrified", "dgkeepsfair")

table4 <- lapply(variables, function(var) {
  out <- tbl4Fxn(baseline, var, "treat_lib")
}) %>%
bind_rows()

```

Once we `xtable` this guy, we get:

|    | variable               | C mean | C SD | T mean | T SD | T-C   | SE   | p-value | C obs  | T obs  |
|----|------------------------|--------|------|--------|------|-------|------|---------|--------|--------|
| 1  | expectedturnout        | 0.79   | 0.24 | 0.80   | 0.23 | 0.01  | 0.02 | 0.73    | 223.00 | 227.00 |
| 2  | secretvote             | 0.67   | 0.26 | 0.65   | 0.25 | -0.02 | 0.02 | 0.45    | 223.00 | 227.00 |
| 3  | candknowsvote          | 0.09   | 0.15 | 0.09   | 0.15 | 0.00  | 0.01 | 0.77    | 223.00 | 227.00 |
| 4  | parliamentaryknowledge | 0.37   | 0.32 | 0.39   | 0.32 | 0.01  | 0.03 | 0.63    | 223.00 | 227.00 |
| 5  | onecandidatevote       | 0.24   | 0.25 | 0.25   | 0.26 | 0.01  | 0.02 | 0.61    | 221.00 | 224.00 |
| 6  | transportproblems      | 0.54   | 0.30 | 0.53   | 0.30 | -0.00 | 0.03 | 0.88    | 219.00 | 225.00 |
| 7  | policesafe             | 0.73   | 0.25 | 0.74   | 0.24 | 0.01  | 0.02 | 0.70    | 223.00 | 227.00 |
| 8  | threatvote             | 0.21   | 0.23 | 0.20   | 0.22 | -0.01 | 0.02 | 0.61    | 223.00 | 226.00 |
| 9  | violenceown            | 0.50   | 0.32 | 0.48   | 0.35 | -0.02 | 0.03 | 0.59    | 219.00 | 226.00 |
| 10 | ownqawmcandidate       | 0.24   | 0.22 | 0.23   | 0.23 | -0.00 | 0.02 | 0.88    | 223.00 | 227.00 |
| 11 | traditionalauthority   | 0.29   | 0.27 | 0.29   | 0.24 | 0.01  | 0.02 | 0.79    | 223.00 | 227.00 |
| 12 | pashtun                | 0.32   | 0.39 | 0.32   | 0.41 | -0.00 | 0.04 | 0.90    | 223.00 | 227.00 |
| 13 | tajik                  | 0.44   | 0.39 | 0.43   | 0.39 | -0.00 | 0.04 | 0.92    | 223.00 | 227.00 |
| 14 | employment             | 0.60   | 0.20 | 0.61   | 0.19 | 0.00  | 0.02 | 0.82    | 223.00 | 227.00 |
| 15 | electrified            | 0.72   | 0.30 | 0.71   | 0.32 | -0.02 | 0.03 | 0.59    | 223.00 | 227.00 |
| 16 | dgkeepsfair            | 0.11   | 0.17 | 0.11   | 0.17 | 0.00  | 0.02 | 0.86    | 223.00 | 225.00 |
| 17 | treat_di_actual        | 0.14   | 0.35 | 0.17   | 0.38 | 0.03  | 0.03 | 0.32    | 233.00 | 238.00 |
| 18 | fefa_visit             | 0.89   | 0.32 | 0.85   | 0.36 | -0.04 | 0.03 | 0.20    | 233.00 | 238.00 |
| 19 | inkproblem             | 0.79   | 0.41 | 0.74   | 0.44 | -0.05 | 0.04 | 0.20    | 233.00 | 238.00 |

A couple of important things to notice from this table: first of all, we successfully replicate all of the numbers from the paper! Secondly, everything is balanced - all of the p-values are well above 0.05. But don't let that convince you by itself: our sample size is quite small, so you want to make sure that T-C is small as well. And lo and behold, it generally is tiny. So we should feel comfortable that this randomization worked.

## The main event

Now that we’ve shown that the randomization worked, we can estimate our treatment effect! Whoo! We want to run a simple regression:

$$\text{VoteDiff} = \alpha + \beta \text{Treat} + \varepsilon$$

This should give us an unbiased estimate of the treatment effect, since treatment was randomly assigned. Beautiful. Let’s try it. First, prepare the data that we’ll need: a variable that’s the absolute value of the difference between the photo and the official records, treatment, and a column of ones.

```
mvdnTC <- read_dta("master_vote_difference_native.dta") %>%
  mutate(absdiff = abs(vote_diff), ones = 1) %>%
  select(pc, pollingstation, absdiff, treat_lib, ones) %>%
  na.omit()
```

Now we can finally run our regression:<sup>14</sup>

```
effect <- OLS(mvdnTC, "absdiff", c("ones", "treat_lib"))
effect

## Source: local data frame [2 x 3]
##
##      betahat      SE      tStatPVal
##      (dbl)      (dbl)      (dbl)
## 1  0.8174969 0.04870143 4.686081e-63
## 2 -0.2357954 0.06137765 1.223351e-04
```

We find that treatment reduces the (absolute value of the) discrepancy between official records and photos of polling center vote postings by 0.235.<sup>15</sup> Note that this “replicates” something that’s not actually reported in Table 6: the difference between column (3) and column (2) in the first row.

TABLE 6—TRIMMED ESTIMATES OF EFFECT ON AGGREGATION FRAUD

|             | Control                              |                                   | Treatment                         |                          |                                                |                                          | Treatment effect          |                           |
|-------------|--------------------------------------|-----------------------------------|-----------------------------------|--------------------------|------------------------------------------------|------------------------------------------|---------------------------|---------------------------|
|             | Mean<br>post-aggreg.<br>votes<br>(1) | Mean<br>abs. vote<br>diff.<br>(2) | Mean<br>abs. vote<br>diff.<br>(3) | Trimming<br>ratio<br>(4) | Trimmed<br>mean<br>$E[Y Y < y_{(1-p)}]$<br>(5) | Trimmed<br>mean<br>$E[Y Y > y_p]$<br>(6) | Lower<br>bound<br>(5)–(2) | Upper<br>bound<br>(6)–(2) |
| Sample:     |                                      |                                   |                                   |                          |                                                |                                          |                           |                           |
| Full sample | 2.318                                | 0.817***<br>(0.210)               | 0.582***<br>(0.140)               | 0.349                    | 0.000<br>(0.245)                               | 0.893***<br>(0.246)                      | –0.817**<br>(0.342)       | 0.076<br>(0.249)          |

## You know what they say about [inferring]

Speaking of statistical significance: there’s actually another way to calculate a  $p$ -value, using a method called *randomization inference*. The idea is pretty simple: we can get exact  $p$ -values without relying on asymptotics, by

<sup>14</sup>We’re going to get the wrong standard errors when we do this. Why? We’re running a regression at the *candidate* level, but treatment was assigned at the *polling center* level. Since you’re now standard error experts from Section 10, what should we do to correct our standard errors? *Hint*: the correct answer starts with “c” and ends with “luster at the polling center level”.

<sup>15</sup>Even when we’re not dealing in standard errors that are conservative enough!

using a permutation test.<sup>16</sup> To do this exact test, all we need to do is randomly generate a bunch of treatment vectors (based on the treatment-control proportions within our sample), and estimate regression coefficients for each draw. At the end of the day, our  $p$ -value is just the percent of these estimated  $\hat{\beta}$ s that our actual estimate is higher than (or lower than, if your real estimate is negative). With our Mad Simulation Skillz®, we can easily do this in R. This will smell familiar to the power calculation stuff we did recently. We'll start by creating a function that creates one treatment vector and outputs a  $\hat{\beta}$ , and then (you guessed it!) run that function a bunch of times using an `apply`.

Importantly for our purposes, treatment is actually assigned at the *polling center* level, so we'll have to deal with that too.

```
randInfer <- function(data, y, treatunit, p) {
  # create a dataset of units at the level of treatment assignment
  units <- distinct_(data, treatunit) %>%
    select_(treatunit)
  # randomly select p% of them to be treated
  random <- randomizer(units, p)
  # create a dataset indicating who's treated
  treatUnits <- mutate(units, treat = random)
  # merge this into your analysis dataset
  myData <- left_join(data, treatUnits, by=treatunit) %>%
    mutate(ones = 1)
  # run the regression
  betahat <- miniOLS(myData, y, c("ones", "treat"))
  return(betahat[2])
}
```

Before we run this a bunch of times, we need to know how many polling centers in our data were treated:

```
treatFrac <- distinct(mvdnTC, pc) %>%
  summarize(mean(treat_lib)) %>%
  as.numeric()

treatFrac

## [1] 0.5503356
```

Okay, now we can run this a bunch of times (we'll do 1,000 for now; the more the better, as with all of this simulation stuff):

```
ndraws <- 1000
rinfOut <- sapply(1:ndraws, function(x) {
  randInfer(mvdnTC, "absdiff", "pc", treatFrac)
}) %>%
  as.tbl_df()
names(rinfOut) <- c("riBhats")
```

---

<sup>16</sup>This was originally proposed by Fisher.

And calculate the  $p$ -value:

```
# effect of interest:
bhat <- as.numeric(effect[2,1])
bigger <- filter(rinfOut, riBhats <= bhat)

pvalue <- nrow(bigger) / ndraws

pvalue

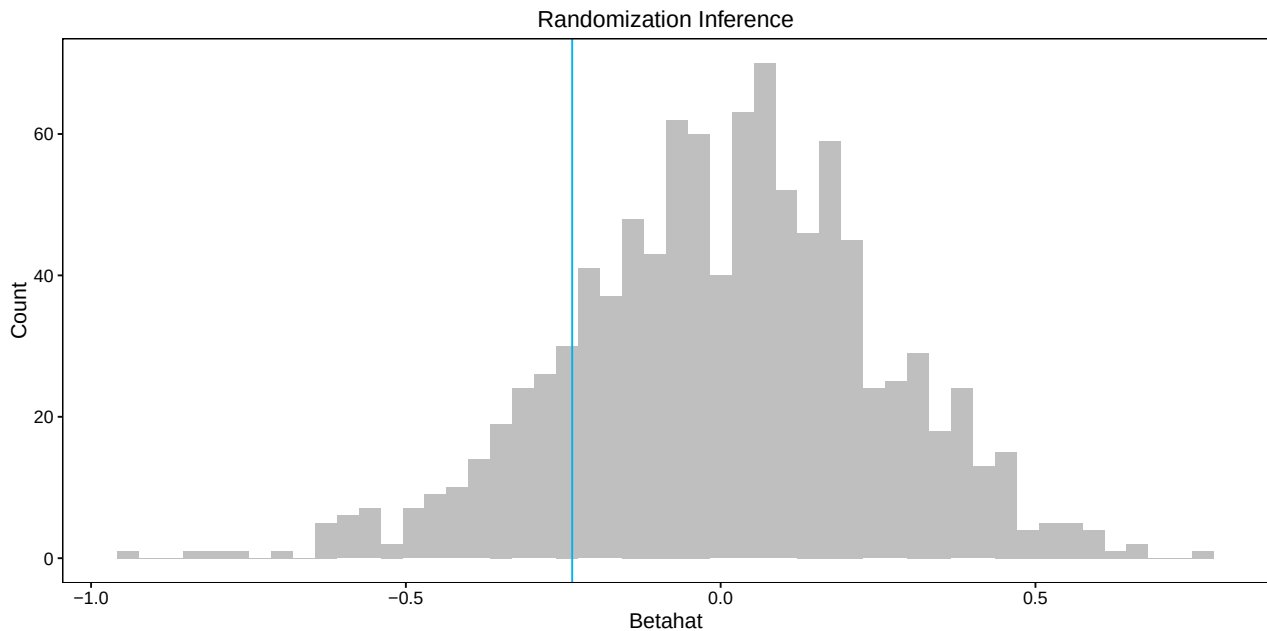
## [1] 0.16
```

This isn't very close to our original  $p$  value. When we do this procedure, it looks like our treatment effect isn't going to be statistically significant at conventional levels. Before crying out that these results are false, though, remember that we weren't clustering correctly - when we do this, the "significant" results that we got before seem to disappear.

This kind of procedure is often really useful for small samples, or generally for convincing referees that your results are real. If you're feeling ambitious, you can plot the RI-based betas, as well as your  $\hat{\beta}$ , like this:

```
myPlot <- ggplot(data = rinfOut, aes(x = riBhats)) +
  geom_histogram(fill = "gray75", color = "NA", bins = 50) +
  geom_vline(xintercept = bhat, color = "deepskyblue2") +
  myThemeStuff + ggtitle("Randomization Inference") +
  xlab("Betahat") + ylab("Count")
```

myPlot



When results really are real, you'll see that blue line *way* to the left of the rest of the distribution.<sup>17</sup>

<sup>17</sup>Check out [this paper](#) by Sol Hsiang and Amir Jina for a nice example, as well as a good discussion of randomization inference in

## Lee[ps and] bounds

But wait, there's more! The authors point out that there's a problem with our estimate of the treatment effect. Remember that we're using only the sample of polling centers where there was both an official vote record *and* a photo of the posted voting results. The issue with this is that treatment actually impacted whether or not polling stations posted results!<sup>18</sup> This is essentially a kind of non-random (non-)attrition: treated units were more likely to appear in the sample than control units. Uh oh. This could definitely cause bias. As is usually the case when we encounter issues in this class, though, a smart econometrician has thought through how to do this. Thanks, David Lee!<sup>19</sup>

The bad news is that in the presence of this non-random selection, we won't be able to recover a treatment effect estimate. The good news is that, with one assumption, we *will* be able to recover *bounds* on that treatment effect - we call them Lee bounds.<sup>20</sup>

How do we do this? We want to remove from our sample the treated units that appeared in the sample *only because they were treated*, because we don't have comparable control units. In order to remove these units, we'll assume that reporting is a monotonic function of treatment.<sup>21</sup> It either has to be the case that the officials who are induced into reporting by the treatment are all posting only when there's no fraud, or are all posting only when there is a *lot* of fraud. In the first case, we want to chop off the treatment units with the smallest outcome variable; in the second case, we want to chop off treatment units with the largest outcome variable. How many do we drop? This is determined by the *trimming ratio*:

$$TR \equiv \frac{\% \text{ Reporting (T)} - \% \text{ Reporting (C)}}{\% \text{ Reporting (T)}}$$

This is easy to compute.<sup>22</sup>

```
mvdnTCTrim <- read_dta("master_vote_difference_native.dta") %>%
  mutate(absdiff = abs(vote_diff), ones = 1) %>%
  select(pc, pollingstation, absdiff, treat_lib, ones) %>%
  group_by(treat_lib)

tcTotal <- summarize(mvdnTCTrim, obs = sum(ones))

tcMiss <- filter(mvdnTCTrim, is.na(absdiff) == FALSE) %>%
  summarize(present = sum(ones)) %>%
  ungroup() %>%
  left_join(tcTotal, by="treat_lib") %>%
  mutate(reportFrac = present / obs,
```

general.

<sup>18</sup>We're not going to replicate that fact, but the authors mention that around 5% more treated units posted vote records than control units, statistically significant at the 10% level.

<sup>19</sup>Original paper [here](#).

<sup>20</sup>This is probably the most frequently implemented procedure for dealing with non-random attrition in modern applied micro papers. I had no idea what it was when I first saw it in Betty's class, so I'm showing it to you now.

<sup>21</sup>This means that either all polling centers are more likely to report election results when treated than when they're not treated, or that all polling centers are less likely to report election results when treated.

<sup>22</sup>I'm following the *code*, rather than the *text* here, in trimming based off of the absolute vote difference rather than the existence of a picture. Technically, the argument in the text suggests that we should be doing this based on the picture - but given that failure to webscrape data for a polling station was essentially random, this should yield very similar results.



```

    nrFrac = 1 - reportFrac)

tcMiss

## Source: local data frame [2 x 5]
##
##   treat_lib present    obs reportFrac    nrFrac
##   (dbl)   (dbl)  (dbl)    (dbl)    (dbl)
## 1       0   17786 183660 0.09684199 0.9031580
## 2       1   30232 203291 0.14871293 0.8512871

tr <- (tcMiss[2, 4] - tcMiss[1, 4]) / (tcMiss[2, 4])
tr

##   reportFrac
## 1  0.3487991

```

Now we can calculate the lower and upper bound of the treatment effect by dropping the upper and lower 0.349% of treatment observations, and comparing the mean absolute vote difference in the control group to the trimmed treatment group.<sup>23</sup>

```

# remove NAs
mvdnTrim <- na.omit(mvdnTCTrim) %>%
  # sort by vote difference
  arrange(absdiff) %>%
  # calculate the percentile of vote difference
  # separately for T vs C (we only care about T)
  mutate(nr = row_number(), total = max(nr),
         pctile = nr / total, trimmer = as.numeric(tr)) %>%
  # remove grouping
  ungroup()

trimmedTreatLow <- filter(mvdnTrim, treat_lib == 1, pctile > trimmer) %>%
  select(-nr, -total, -pctile, -trimmer)

trimmedTreatHigh <- filter(mvdnTrim, treat_lib == 1, pctile <= (1-trimmer)) %>%
  select(-nr, -total, -pctile, -trimmer)

controlOnly <- na.omit(mvdnTCTrim) %>%
  filter(treat_lib == 0)

```

Now that we've done this trimming, we're ready to calculate our lower and upper bound estimates, which is easy: take the mean of the treated observations and subtract off the mean of the control observations.

<sup>23</sup>Again, here we're doing something to be comparable to the code, but which I actually think is *wrong*. Notice that in the below code, we're trimming based off an individual *row's* percentile ranking, where rows are polling substation-candidates. But what does this do? It means that we could end up trimming part of a substation, but not the other part. Given that we're supposed to be trimming because substations didn't put up vote lists to take pictures of, as a result of treatment, this seems incorrect. If I were writing this paper, I'd base my percentiles on *polling substation* (or even polling center) data.

```
## UPPER BOUND
upperBound <- bind_rows(controlOnly, trimmedTreatLow) %>%
  group_by(treat_lib) %>%
  summarize(mean(absdiff)) %>%
  ungroup()
upperBound

## Source: local data frame [2 x 2]
##
##   treat_lib mean(absdiff)
##   (dbl)      (dbl)
## 1      0      0.8174969
## 2      1      0.8932345

ub <- upperBound[2,2] - upperBound[1,2]

## LOWER BOUND
lowerBound <- bind_rows(controlOnly, trimmedTreatHigh) %>%
  group_by(treat_lib) %>%
  summarize(mean(absdiff)) %>%
  ungroup()

lowerBound

## Source: local data frame [2 x 2]
##
##   treat_lib mean(absdiff)
##   (dbl)      (dbl)
## 1      0      0.8174969
## 2      1      0.0000000

lb <- lowerBound[2,2] - lowerBound[1,2]

lb

##   mean(absdiff)
## 1    -0.8174969

ub

##   mean(absdiff)
## 1    0.07573755
```

Note that we're not calculating standard errors for this - that's because to do that correctly, you'd need to bootstrap this entire procedure.<sup>24</sup> That aside, though, we've now replicated the first row of Table 6!

---

<sup>24</sup>We'll leave that as an exercise for the reader.

That's it for ARE 212 section! Thanks for putting up with me all semester - I've had a lot of fun, and learned a few things along the way; I hope you have too!